

BASICS OF C

Overview • Program Structure • Identifiers • Keywords
Variables • Data Types • Formatted Input / Output

A Detailed Study Guide with Definitions, Diagrams & Worked Examples

Course	BCSG 0802 : Computer Programming-I
Module	II — Basics of C
Language	C (ANSI C / C99)
Level	Beginner — First-year B.Tech

Contents

#	Topic	What you will learn
1	Overview of C	What C is, its history, features & how a C program runs
2	Structure of a C Program	The six sections + a fully annotated example
3	Identifiers	Naming rules, valid vs. invalid names
4	Keywords	The 32 reserved words of C
5	Variables	Declaration, initialisation & the memory model
6	Data Types	Basic, derived & user-defined types, sizes & ranges
7	Formatted Input / Output	printf(), scanf(), format specifiers & escapes
8	Quick Revision & Common Mistakes	One-page summary you can memorise

1. Overview of C

Definition

C is a general-purpose, procedural, structured programming language developed by **Dennis Ritchie** at **Bell Laboratories** in **1972**. It is called a **middle-level language** because it combines the features of high-level languages (easy to read/write) with the power of low-level languages (direct access to memory and hardware).

1.1 A Short History

C did not appear out of nowhere — it evolved from earlier languages. Each step borrowed ideas and fixed limitations of the one before it:

Year	Language	Created by	Note
1960	ALGOL	Committee	Root of modern structured languages
1967	BCPL	Martin Richards	“Basic Combined Programming Language”
1970	B	Ken Thompson	Used to develop early UNIX
1972	C	Dennis Ritchie	Added data types & more
1989	ANSI C (C89)	ANSI committee	First official standard
1999	C99	ISO	Added <code>//</code> -comments, new types
2011	C11 / C17	ISO	Multithreading, refinements

1.2 Why Learn C? (Key Features)

- **General-purpose:** used for operating systems, embedded devices, games, databases and compilers.
- **Procedural & structured:** a program is built from functions and clear control structures (if, for, while).
- **Fast & efficient:** compiles to native machine code that runs very close to the hardware.
- **Portable:** a well-written C program can be re-compiled on many machines with little or no change.
- **Rich standard library:** ready-made functions like `printf`, `scanf`, `strlen`.
- **Powerful:** pointers give direct access to memory; the UNIX operating system itself was written in C.
- **Foundation language:** C++, Java, C#, JavaScript and Python all borrow C's syntax — learning C makes the rest easier.

1.3 How a C Program Runs — The Compilation Process

A computer's CPU cannot understand C directly — it only understands **machine code** (1s and 0s). Your **source code** must therefore pass through several tools before it becomes a runnable program. This chain is called the **compilation process**:

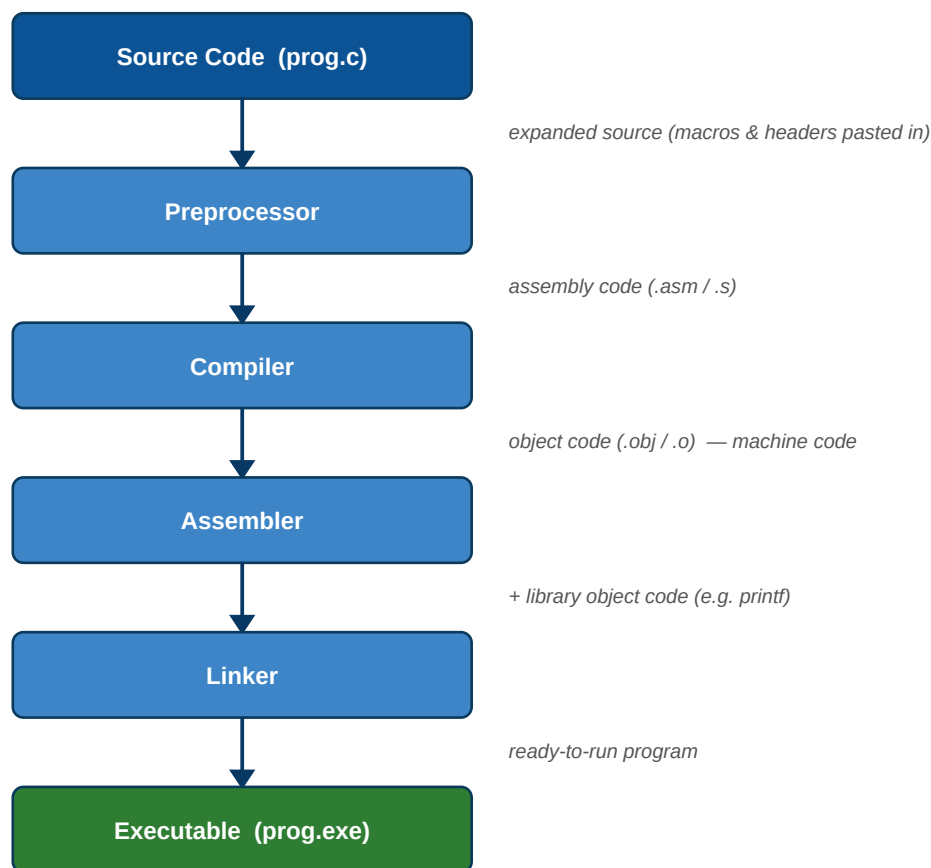


Figure 1: The four tools — preprocessor, compiler, assembler and linker — turn `prog.c` into an executable.

- **Preprocessor** — handles lines starting with `#` (e.g. `#include`, `#define`): it pastes in header files and expands macros.
- **Compiler** — translates the expanded C code into **assembly language**.
- **Assembler** — converts assembly into **object code** (`.obj` / `.o`).
- **Linker** — joins your object code with library code (like the code for `printf`) to produce the final **executable**.

1.4 Your First C Program

```

#include <stdio.h>          /* bring in input/output functions */

int main(void)              /* execution begins here */
{
    printf("Hello, World!\n"); /* print a message on screen */
    return 0;                /* tell the OS the program ended OK */
}
  
```

Listing 1: The classic “Hello, World!” program.

Output ▼

Hello, World!

Note

Every C statement ends with a **semicolon** `;`. C is **case-sensitive**: `main` and `Main` are two different names.

2. Structure of a C Program

Every C program follows the same general skeleton. It can be divided into **six sections**. Only the `main()` function is compulsory — the rest are used when needed.

1. Documentation Section comments / author / purpose	Optional
2. Link / Preprocessor Section <code>#include <stdio.h></code>	Common
3. Definition Section <code>#define PI 3.14159</code>	Optional
4. Global Declaration Section variables & prototypes	Optional
5. main() Function Section execution starts here { ... }	MANDATORY
6. Sub-program / Function Section user-defined functions	Optional

Figure 2: The six sections of a C program, in the order they usually appear.

2.1 The Six Sections Explained

Section	Purpose	Example
Documentation	Comments describing the program (ignored by compiler)	<code>/* Author, date, purpose */</code>
Link / Preprocessor	Includes header files & libraries	<code>#include <stdio.h></code>
Definition	Defines symbolic constants & macros	<code>#define PI 3.14159</code>
Global Declaration	Global variables & function prototypes	<code>float area(float r);</code>
main() function	The starting point of execution (MANDATORY)	<code>int main(void) { ... }</code>
Sub-programs	User-defined functions	<code>float area(float r){...}</code>

2.2 A Fully Annotated Example

The program below uses **every** section. Read the comments — they map each line back to Figure 2.

```
/* ----- 1. Documentation: find area of a circle ----- */
#include <stdio.h>           /* 2. Link section      */
#define PI 3.14159          /* 3. Definition       */

float radius = 5.0;         /* 4. Global variable  */
float area(float r);        /* function prototype */

int main(void)              /* 5. main() section   */
{
    float a;
    a = area(radius);
    printf("Area of circle = %.2f\n", a);
    return 0;
}

float area(float r)          /* 6. Sub-program      */
{
    return PI * r * r;
}
```

Listing 2: One program touching all six sections.

Output ▼

Area of circle = 78.54

2.3 Rules to Remember

- Execution **always** begins at `main()`.
- Statements end with a semicolon `;`.
- A block of code is grouped inside curly braces `{ }`.
- Comments `/* ... */` (multi-line) and `// ...` (single-line) are ignored by the compiler.
- C is free-form: indentation is for *humans*, not the compiler — but good indentation is essential for readable code.

3. Identifiers

Definition

An **identifier** is the **name** given to a program element such as a variable, function, array, or structure. It lets you refer to that element throughout your program.

For example, in `int marks;` the word `marks` is an identifier that names an integer variable.

3.1 Rules for Naming Identifiers

- May contain **letters** (a–z, A–Z), **digits** (0–9) and the **underscore** `_`.
- The **first character** must be a letter or an underscore — **never a digit**.
- **No spaces** and **no special symbols** (`@ # $ % . -` etc.) are allowed.
- A **keyword** (see Section 4) cannot be used as an identifier.
- Identifiers are **case-sensitive**: `Age`, `age` and `AGE` are three different names.
- There is no strict length limit, but only the first **31 characters** are usually significant.

3.2 Valid vs. Invalid Identifiers

Identifier	Valid?	Reason
<code>total_marks</code>	✓ Valid	letters + underscore
<code>_count</code>	✓ Valid	may start with underscore
<code>student2</code>	✓ Valid	digits allowed (not first)
<code>avgScore</code>	✓ Valid	mixed case is fine
<code>2ndrank</code>	✗ Invalid	starts with a digit
<code>total marks</code>	✗ Invalid	contains a space
<code>net-salary</code>	✗ Invalid	'-' is not allowed
<code>float</code>	✗ Invalid	'float' is a keyword
<code>first#name</code>	✗ Invalid	'#' is a special symbol

Note

Good practice: choose *meaningful* names. `totalMarks` is far clearer than `tm` or `x`.

4. Keywords

Definition

A **keyword** is a **reserved word** that already has a special, fixed meaning in C. Keywords are the building blocks of the language and **cannot** be used as identifiers (variable or function names).

Standard **ANSI C (C89)** defines exactly **32 keywords**. All of them are written in **lowercase**:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Table: The 32 keywords of ANSI C.

Note

Later standards added a few more (e.g. C99 added `_Bool`, `inline`, `restrict`; C11 added `_Atomic` and others), but for this course the **32 classic keywords** are what you must know.

4.1 Keywords vs. Identifiers

Keyword	Identifier
Predefined meaning fixed by the language	Meaning given by the programmer
Cannot be changed or redefined	Can be created freely (within the rules)
Always lowercase	Any valid combination of letters/digits/_
Example: <code>int</code> , <code>for</code> , <code>while</code>	Example: <code>age</code> , <code>total</code> , <code>main</code>

5. Variables

Definition

A **variable** is a **named location in memory** used to store a value that can **change** while the program runs. The name is the identifier; the value is what the box currently holds.

5.1 The Memory Model

When you declare a variable, the compiler reserves a block of memory, gives it an **address**, and links your chosen **name** to it. You put values in and read them out using the name — you never have to remember the address.

```
int age = 25;
```

creates a labelled box in memory:

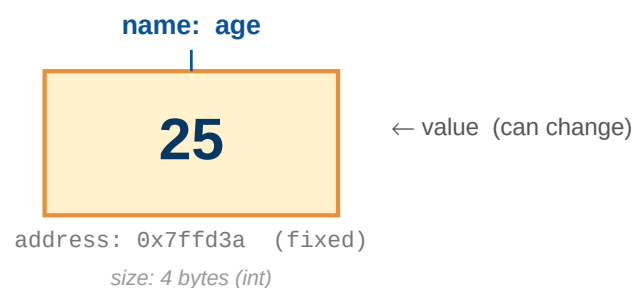


Figure 3: `int age = 25;` — the name `age` labels a 4-byte box holding the value 25.

5.2 Declaration, Initialisation & Assignment

- **Declaration** — tells the compiler the variable's *type* and *name*, reserving memory: `int age;`
- **Initialisation** — giving a value *at the moment of declaration*: `int age = 25;`
- **Assignment** — giving (or changing) a value *later*: `age = 30;`

```
int age;           /* 1. declaration only          */
age = 25;          /* 2. assignment (value stored) */

float salary = 45000.50; /* declaration + initialisation */
char grade = 'A';       /* a single character in quotes */

int a = 10, b = 20, c = 30; /* declare many at once      */

const float PI = 3.14159; /* const = value cannot change */
```

Listing 3: Different ways to create variables.

5.3 Rules for Variable Names

A variable name **is** an identifier, so it follows exactly the identifier rules from Section 3: start with a letter/underscore, no spaces or special symbols, and not a keyword.

5.4 Types of Variables (by Scope)

Type	Where declared	Where usable
Local variable	Inside a function/block	Only within that function/block

Type	Where declared	Where usable
Global variable	Outside all functions	Everywhere in the program
Constant	With the const keyword or #define	Value fixed — cannot be modified

Note

A **constant** is like a variable whose value is locked. Use it for values that must never change, such as PI or a tax rate.

6. Data Types

Definition

A **data type** tells the compiler **what kind of value** a variable can hold (whole number, decimal, character, ...) and **how much memory** to reserve for it. Choosing the right type saves memory and prevents errors.

6.1 Classification of Data Types

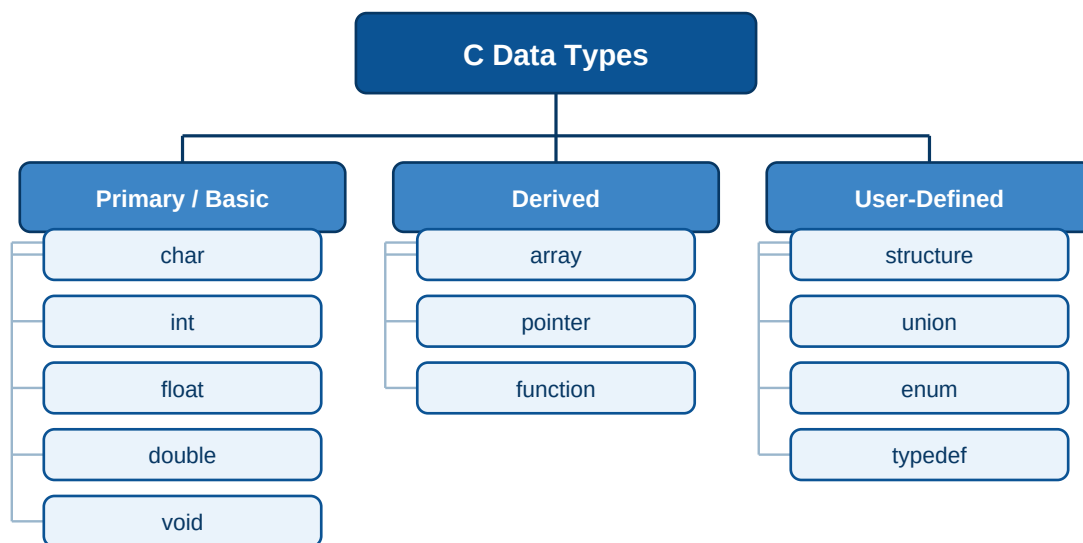


Figure 4: C data types are grouped into Primary (basic), Derived and User-Defined types.

- **Primary / Basic:** char, int, float, double, void — built into the language.
- **Derived:** array, pointer, function — built *from* the basic types.
- **User-Defined:** structure, union, enum, typedef — types *you* create.

6.2 The Basic Data Types — Size, Range & Format

Sizes below are for a typical **modern 64-bit compiler (GCC)**. The exact size can vary between machines — that is why C is called machine-dependent for sizes.

Data Type	Size	Format	Typical Range
char	1 byte	%c	−128 to 127 (stores one character)
unsigned char	1 byte	%c	0 to 255
short int	2 bytes	%hd	−32,768 to 32,767
int	4 bytes	%d	−2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	%u	0 to 4,294,967,295

Data Type	Size	Format	Typical Range
long int	4/8 bytes	%ld	at least ± 2.1 billion
long long int	8 bytes	%lld	$\pm 9.2 \times 10^{18}$
float	4 bytes	%f	$\pm 3.4 \times 10^{38}$ (6–7 digits)
double	8 bytes	%lf	$\pm 1.7 \times 10^{308}$ (15–16 digits)
long double	12/16 bytes	%Lf	even larger precision
void	0 bytes	—	'no value' / empty type

6.3 Type Modifiers

Keywords `signed`, `unsigned`, `short` and `long` change the size or range of the basic types:

- **signed** (default) — can hold negative *and* positive values.
- **unsigned** — only zero and positive values, so the positive range doubles.
- **short / long** — make an integer take less / more memory.

6.4 Checking Sizes with `sizeof`

The `sizeof` operator reports how many bytes a type uses on *your* machine:

```
#include <stdio.h>

int main(void)
{
    printf("char      : %d byte\n", (int)sizeof(char));
    printf("int       : %d bytes\n", (int)sizeof(int));
    printf("float     : %d bytes\n", (int)sizeof(float));
    printf("double    : %d bytes\n", (int)sizeof(double));
    printf("long long : %d bytes\n", (int)sizeof(long long));
    return 0;
}
```

Listing 4: Measuring type sizes at run time.

Output ▼

```
char      : 1 byte
int       : 4 bytes
float     : 4 bytes
double    : 8 bytes
long long : 8 bytes
```

Note

On old **16-bit Turbo C**, an `int` is only **2 bytes**. Always test with `sizeof` instead of assuming a size.

7. Formatted Input / Output

Definition

Formatted I/O means reading and displaying data in a controlled format using a **format string**. The two work-horse functions are `printf()` (output to screen) and `scanf()` (input from keyboard). Both live in `<stdio.h>`.

7.1 printf() — Formatted Output

`printf` prints text to the screen. Inside the quoted **format string** you place **format specifiers** (like `%d`) that are replaced, in order, by the values you list after the string:

```
printf("format string", value1, value2, ...);

printf("My age is %d and marks %.1f\n", 21, 87.5);
/*      ↑           ↑↑
   %d  is replaced by 21
   %.1f is replaced by 87.5 (1 decimal place)      */
```

Listing 5: How printf maps specifiers to values.

Common Format Specifiers

Specifier	Used for	Example value
<code>%d</code> or <code>%i</code>	int (decimal integer)	42
<code>%u</code>	unsigned int	42
<code>%f</code>	float / double	3.140000
<code>%.2f</code>	float with 2 decimals	3.14
<code>%lf</code>	double (in <code>scanf</code>)	3.14159
<code>%c</code>	single character	'A'
<code>%s</code>	string (text)	Hello
<code>%ld</code> / <code>%lld</code>	long / long long int	1000000
<code>%x</code> / <code>%o</code>	hexadecimal / octal	2a / 52
<code>%e</code>	scientific notation	3.14e+00
<code>%p</code>	memory address (pointer)	0x7ffd3a
<code>%%</code>	a literal percent sign	%

Escape Sequences

An **escape sequence** is a backslash `\` followed by a character. It represents something that is hard to type directly, such as a new line.

Sequence	Meaning	Sequence	Meaning
<code>\n</code>	new line	<code>\t</code>	horizontal tab
<code>\\</code>	a backslash	<code>\"</code>	double quote
<code>\'</code>	single quote	<code>\0</code>	null character

Sequence	Meaning	Sequence	Meaning
\r	carriage return	\b	backspace
\a	alert / beep	\f	form feed

Controlling Width & Precision

```
#include <stdio.h>

int main(void)
{
    int    age    = 21;
    float  pi     = 3.14159f;
    char   grade  = 'A';

    printf("Age    = %d\n",    age);    /* plain integer      */
    printf("Pi     = %.2f\n",  pi);     /* 2 decimal places  */
    printf("Grade = %c\n",    grade);   /* one character      */
    printf("[%5d]\n",    age);          /* width 5, right-aligned */
    printf("[%5d]\n",    age);          /* width 5, left-aligned  */
    return 0;
}
```

Listing 6: Width and precision control.

Output ▼

```
Age    = 21
Pi     = 3.14
Grade = A
[   21]
[21   ]
```

7.2 scanf() — Formatted Input

scanf reads values typed by the user and stores them in variables. It needs the **address** of each variable, given with the **address-of operator &**:

```
#include <stdio.h>

int main(void)
{
    int    roll;
    float  marks;
    char   section;

    printf("Enter roll, marks and section: ");
    scanf("%d %f %c", &roll, &marks, &section);    /* note the & */

    printf("Roll = %d, Marks = %.1f, Section = %c\n",
           roll, marks, section);
    return 0;
}
```

Listing 7: Reading three different types with scanf.

Output ▼

```
Enter roll, marks and section: 7 88.5 B
Roll = 7, Marks = 88.5, Section = B
```

Common Mistakes to Avoid

- **Forgetting the & in scanf** — `scanf("%d", n);` is wrong; it must be `scanf("%d", &n);`
- **Wrong specifier** — using `%d` for a `float` prints garbage. Match the type.
- **Using `%f` for double in scanf** — `scanf` needs `%lf` for a `double`.
- **Missing `\n`** — without it, output lines run together.

8. Quick Revision & Common Mistakes

8.1 One-line Summary of Each Topic

Topic	Remember this
Overview	C = middle-level, procedural language by Dennis Ritchie (1972).
Compilation	Source → Preprocess → Compile → Assemble → Link → Executable.
Structure	6 sections; only main() is mandatory; statements end with ;
Identifiers	Names; start with letter/_, no spaces, case-sensitive.
Keywords	32 reserved words; cannot be used as identifiers.
Variables	Named memory box; declare, then initialise/assign.
Data types	int, float, char, double, void + derived + user-defined.
printf/scanf	printf shows values; scanf reads them (needs &).

8.2 Top Beginner Mistakes

- Missing semicolon ; at the end of a statement.
- Forgetting & in scanf.
- Mismatched format specifier and variable type.
- Using a keyword (like int) as a variable name.
- Confusing = (assignment) with == (comparison).
- Forgetting #include <stdio.h> before using printf/scanf.

8.3 Practice Questions

- Write a program that reads two integers and prints their sum, difference and product.
- Which of these are valid identifiers: _total, 2marks, net salary, Grade1?
- State the size and format specifier for int, float, char and double.
- Write a program using scanf to read your name, age and marks, then print them neatly with printf.

End of Notes — Basics of C • BCSG 0802